

Nuclear Power Plant Life-Cycle Digital Twin

A Formalized Model for Artificial Intelligence
and Machine Learning Applications

Yuriy Dmitrishin

yu.dmitrishin@gmail.com

<https://dmitrishin.github.io>

Summary based on the original research paper:

Formalization of a Nuclear Power Plant Life-Cycle Model for AI and Machine Learning Applications

Author: Yuriy Dmitrishin

Year: 2025

DOI: <https://doi.org/10.5281/zenodo.18527396>

GitHub repository: <https://github.com/dmitrishin/npp-lifecycle-ai-ml-model>

1 Purpose

The model is a digital twin of the nuclear power plant life cycle: a formalized digital representation of the entire project covering the complete life cycle from engineering and construction through commissioning, operation, and decommissioning.

The purpose of the model is to describe and simulate the NPP life cycle while simultaneously satisfying three requirements:

- **Universality of modeling.**

The model must be sufficiently universal to represent any life-cycle process, including engineering, procurement, construction and installation activities, commissioning, operation, and decommissioning, without restrictions on the type or complexity of the processes being modeled.

- **Suitability for AI algorithms.**

The model is structured so that modern artificial intelligence and machine learning algorithms can operate on it, including graph neural networks, large language models, optimization methods, and forecasting algorithms. Data are represented in formats suitable for training and analysis.

- **Comprehensibility for specialists.**

The model remains understandable to engineers, project managers, and domain experts.

Every element, relationship, and metric has a clear substantive meaning. Algorithmic results can be explained and verified by humans.

The model is not tied to any particular algorithm or technology. Its architecture provides for modular components and replaceable AI algorithms. As new methods emerge, including new neural-network architectures, more capable language models, and other approaches, they can be integrated without restructuring the underlying model.

The model evolves in two directions: together with the actual NPP project, reflecting changes in configuration, project progress, and data; and as an analytical instrument, expanding its analytical and predictive capabilities as AI technologies develop.

Unlike digital twins of individual physical objects such as equipment or buildings, the object represented here is the life cycle as a whole. The model captures not only the physical and functional configuration of the plant, but also processes, dependencies, readiness stages, project documentation, and quantitative indicators associated with the creation and operation of the NPP.

As the project progresses, the model evolves together with the actual facility and maintains an up-to-date representation of its current state.

The model is not itself a software system. It is a detailed specification of the data architecture of a digital twin that can be adapted, simplified, or extended for particular tasks and projects.

The underlying principles can also be applied to other complex technical systems.

2 Architecture: Five Components

The model consists of five interconnected components, each responsible for a specific aspect of project representation. Together they provide a comprehensive digital representation of the project.

- **M — Structure.**

The hierarchical structure of the project: systems, subsystems, equipment, activities, and relationships between them across all levels of detail.

- **S — Semantics.**

The conceptual framework of the project: terms, definitions, relationships between concepts, and terminology rules.

- **D — Corpus.**

Links to documentation and data. Each model element can be traced to a specific source, such as a design document, protocol, report, record, or dataset.

- **Q — Metrics.**

Quantitative characteristics such as readiness, cost, risk, and schedule. These include computable indicators used for analysis and decision-making.

- **U — Meta-Level.**

Management and coordination of all components, AI model training processes, and data-quality control.

The components operate jointly while remaining independent. A change in one component does not require restructuring the others.

This provides flexibility and enables phased deployment of the model, from a basic structural representation to full integration of all five layers.

3 Hierarchical Graph: Structural Foundation

The model is based on a multi-level graph in which every project object, such as a system, building, equipment item, or work stage, is represented as an element or vertex, while relationships between objects are represented as edges.

The graph forms the structural backbone of the model onto which all other components are layered.

Two principal types of relationships are used:

- **Horizontal relationships** represent dependencies between objects at the same hierarchical level.

For example, installation of a pipeline may depend on installation of its supports. A single relationship may connect one source element to several dependent objects where multiple activities depend on one decision or condition.

- **Vertical relationships** represent part-whole relationships between hierarchical levels.

For example, a reactor system can be decomposed into the reactor vessel, steam generators, reactor coolant pumps, and other components.

Decomposition and aggregation.

Any element at a higher level can be decomposed into lower-level elements down to individual equipment items or activities.

Conversely, information from lower levels can be automatically aggregated upward. For example, the readiness of individual systems can be aggregated into the overall readiness of a commissioning complex.

Correctness guarantees.

The model includes built-in rules, or invariants, that ensure:

- absence of cyclic dependencies within each hierarchical level;
- assignment of every element to exactly one hierarchical level;
- integrity of relationships between hierarchical levels.

These rules can be checked automatically and prevent logical inconsistencies in the model.

4 Classification System

Every model element, whether an object or a relationship, can be assigned to one or more classes.

Classes are organized into hierarchical trees from general categories to more specific categories.

This structure is simultaneously understandable to engineers, for example:

equipment $\rightarrow$ heat-exchange equipment $\rightarrow$ steam generator,

and provides a formal basis for machine learning algorithms that use classes as features.

The model supports multiple classification. A single element can belong to several classes simultaneously, reflecting different roles.

For example, an equipment item can be classified simultaneously by equipment type, safety system, and commissioning complex.

Selection of an optimal classification.

For a particular analytical scenario, the model allows the most appropriate interpretation to be selected from the set of admissible classifications.

The space of all admissible interpretations is formalized, enabling the selection problem to be addressed using machine learning methods.

If no consistent interpretation is possible, the condition is automatically identified as a data error. This provides an additional data-quality control mechanism.

5 Typed Attributes and Aggregation

Every element of the model has a set of attributes. These may be numerical, textual, logical, or categorical.

Each attribute is strictly typed, and the system verifies automatically that its value belongs to the permitted domain.

For specialists, these are familiar characteristics such as mass, cost, schedule, and readiness status. For algorithms, they form typed features suitable for training and analysis.

Automatic aggregation.

Attribute values from lower hierarchical levels can be automatically aggregated upward according to defined rules, including:

- **summation** — for example, the cost of a complex as the sum of the costs of its components;
- **minimum** — for example, system readiness determined by the readiness of its least-ready element;
- **average** — for example, average activity duration;
- **logical aggregation** — for example, a system is considered ready only when all required subsystems are ready.

Aggregated characteristics can also serve as the basis for derived classifications, such as automatically assigning an element to a high-risk category based on its quantitative indicators.

6 Life-Cycle Model: Core of the Framework

The life cycle is the central object of the digital twin.

Unlike static models describing a facility at a particular moment, this model represents the entire

NPP life cycle as a sequence of phases, with each phase described by its own model subgraph.

The life-cycle representation has the following properties:

- **Complete coverage.**

Every model element belongs to at least one life-cycle phase.

- **Controlled overlap.**

An element may belong to no more than two adjacent phases, reflecting actual transition processes, but cannot skip an intermediate phase.

- **Formalized transitions.**

Transitions between phases are defined using dedicated transition sets and conditions implemented through decision nodes.

The phases include both physically realized states and activities that have no physical embodiment, such as engineering and procurement.

The model is therefore applicable to the entire life cycle, from initial engineering decisions through decommissioning.

7 Commissioning Complexes and Readiness Assessment

The model includes specialized mechanisms for representing commissioning complexes: autonomous groups or subsystems of an NPP that are brought into service sequentially.

Physical and functional realization.

Each element can be represented through two related states:

- **physically realized** — installed or constructed, but not necessarily operational;
- **functionally active** — physically realized and capable of performing its intended functions.

Functional activity is impossible without physical realization.

Functional readiness is evaluated on a continuous scale from 0 to 1, where 1 represents full readiness.

System readiness is determined by the minimum readiness of its required components. A system is therefore not considered ready if at least one necessary subsystem remains unready.

Stage filtering.

Each commissioning complex passes through a strictly increasing sequence of readiness stages from 0 to 1.

Transition to the next stage is permitted only when formal conditions are satisfied, such as physical completeness and achievement of the required readiness level.

The overall commissioning status is determined by aggregation of the stages of all commissioning complexes.

Shared elements.

The model also accounts for components required by several commissioning complexes, including infrastructure and control systems.

Such elements may be introduced with the first relevant commissioning complex and remain active for subsequent complexes.

8 Semantic Layer

The semantic layer is a linguistic and conceptual model of the NPP life cycle.

It has a dual function: it keeps the model understandable to humans while also providing a formal conceptual and categorical framework that supports complex reasoning and logical inference.

The semantic layer manages:

- terms and definitions, including synonyms, abbreviations, and multilingual terminology;
- typed relationships between terms, such as “part of”, “precedes”, “is a subclass of”, and “is located in”;
- categorization of terms, for example activity, milestone, equipment, or document;
- built-in integrity checks, such as prevention of cycles in part–whole hierarchies.

The semantic layer is linked to the structural model through bidirectional mappings.

Structural elements can be associated with terms, while terms can in turn be mapped to corresponding structural elements.

This enables grouping of elements by conceptual categories, analysis of the effects of terminology changes, and automatic enrichment of annotations.

9 Document Corpus and Traceability

The model provides formal links between model elements and data sources, including engineering documentation, test records, protocols, standards, and databases.

Each document receives a unique identifier, type, and timestamp.

A system of semantic references makes it possible to link not only to an entire document but also to a specific fragment, such as a section, table, or paragraph.

When a document changes, a new version with a new identifier is created, providing a continuous history of changes.

Documentation criterion.

The model is considered fully documented when every model element can be traced to at least one source.

The corpus is organized through a system of independent classifications, for example by subject, document type, or status.

Each classification can have its own hierarchical structure and can evolve independently from the others.

10 Schedule Planning and Analysis

The model includes built-in support for network-planning methods:

- **PERT** — probabilistic schedule estimation using optimistic, most likely, and pessimistic duration estimates, together with critical-path and float calculations;
- **GERT** — stochastic modeling with probabilistic transitions, allowing analysis of alternative scenarios and processes with uncertain outcomes;
- **PDM** — precedence diagramming with four relationship types: finish-to-start, start-to-start, finish-to-finish, and start-to-finish, including time lags.

All three methods are integrated into the common model structure.

The results of schedule analysis update model-element attributes and participate in the aggregation of higher-level indicators.

11 Integration with Coding Systems: KKS and RDS-PP

The model can be formally integrated with industry coding and designation systems for power-plant systems and equipment, including KKS and RDS-PP.

The code assigned to each element can be used to determine automatically:

- its hierarchical level based on the structure of the code;
- its association with systems and subsystems through prefixes and designation blocks;
- its classification based on the positional semantics of the code.

When a structured coding system such as KKS is available, hierarchy, classification, and aggregation can be derived automatically, significantly simplifying model population and validation.

12 Numerical Representation for AI Algorithms

One of the key properties of the model is its automatic transformation into numerical representations suitable as standard inputs for machine learning algorithms.

The original model remains in a human-readable form as a graph containing named elements, classes, relationships, and attributes.

Numerical representations can be generated from it automatically, including:

- an **adjacency matrix**, representing which elements are connected;
- a **graph Laplacian**, supporting analysis of structural graph properties;
- a **reachability matrix**, representing which elements can influence or reach other elements;
- a **distance matrix**, representing structural distance between elements;
- a **stochastic transition matrix**, supporting probabilistic process modeling.

A global block matrix combines all hierarchical levels into a unified representation while preserving both horizontal and vertical relationships.

Different storage formats can be used, ranging from dense to sparse representations.

The model therefore operates in two mutually consistent forms: as a human-readable graph with meaningful domain elements, and as a set of numerical matrices suitable for computational algorithms.

13 Meta-Level and AI Support

The meta-level acts as the coordinator of the model.

It coordinates the operation of all five components and supports:

- aggregation and filtering of data for analytical tasks;
- generation of new model elements with automatic correctness checks;
- formation of training datasets for machine learning models;
- training and evaluation of AI models with quality control based on metrics;
- an adaptive cycle of generation, training, evaluation, and improvement.

The meta-level is open to replacement and extension of algorithms.

AI algorithms are separated into categories such as aggregation, generation, training, and evaluation and interact with the model through a common interface.

This makes it possible, for example, to replace a schedule-forecasting algorithm with a more advanced one without modifying the other model components.

Specific algorithms can change while the underlying architecture remains stable.

All meta-level operations are required to preserve model integrity. The result of every permissible transformation remains a valid configuration satisfying the defined model rules.

14 Change Management

The digital twin is not a static snapshot but an evolving model.

Any change in structure, classification, attributes, or semantics is formalized as a permissible transformation that moves the model from one valid state to another.

The set of transformations forms a closed system:

- any sequence of permissible changes remains permissible;
- no change is itself a permissible state;
- every change is validated using a common set of rules.

This provides configuration control and reduces the risk of data inconsistency during parallel work.

The model remains in a consistent state while reflecting the current project configuration.

Dedicated decision nodes in the graph represent points at which project structure can change, including phase transitions, configuration revisions, and the inclusion or exclusion of equipment.

15 Two Development Vectors

The model development strategy provides for expansion in two independent but complementary directions.

15.1 Vector 1: Detail — Increasing Depth

The model can be progressively detailed along three dimensions.

- **Processes.**

Representation can progress from high-level life-cycle phases to detailed technological processes, individual activities, and control points.

Each level of detail can be decomposed into lower levels using the general decomposition mechanism.

- **The NPP as a technical object.**

Representation can progress from the plant or unit as a whole to systems, subsystems, individual equipment items, assemblies, and components.

The required level of detail depends on the task. High-level representation may be sufficient for strategic planning, whereas commissioning activities may require representation down to individual components.

- **Management domains.**

Representation can progress from overall project status to individual management domains, including schedule, cost, quality, risk, procurement, configuration, and documentation management.

Each domain can be developed to the required level of detail independently from the others.

Increasing detail does not require restructuring the model because aggregation provides automatic reconstruction of higher-level indicators from lower-level data.

15.2 Vector 2: Expansion — Increasing Breadth

The model can be expanded functionally by integrating new algorithms and covering additional application areas.

Examples include expansion from schedule forecasting and risk assessment to:

- procurement optimization;
- automated quality control of documentation;
- predictive maintenance;
- support for regulatory decision-making.

The analytical toolkit can also be extended through new metrics, more advanced aggregation methods, integration with external information systems, and expansion of the semantic layer.

The two development vectors are independent and complementary.

The greatest effect is achieved when both progress simultaneously: more detailed representation provides richer information for algorithms, while more capable algorithms increase the value of detailed modeling.

16 Directions for AI Application

Artificial intelligence technologies are used not only to solve application tasks on an already populated model, but also to create, populate, and develop the model itself.

Four principal directions are defined.

16.1 Direction 1: Model Development and Implementation

AI algorithms can support improvement of the model architecture and its implementation in software.

Possible applications include:

- generation and verification of code implementing model components;
- automatic consistency checking, including validation of invariants and detection of errors;
- optimization of model structure and recommendations for task-specific configurations;
- adaptation to a specific project, including generation of templates and initial configurations based on reference or typical projects.

16.2 Direction 2: Model Population with Data

A full model requires project-specific data.

AI algorithms can operate across the principal information layers.

Structure (M).

AI can support:

- automatic construction of NPP object hierarchies from engineering documentation and specifications;
- identification of dependencies between elements by analyzing text, schedules, tables, and other project information;
- classification of elements;
- detection of gaps and inconsistencies in the structural model.

Semantics (S).

AI can support:

- extraction of terms and concepts from documentation and standards;
- construction of relationships between concepts;
- creation and maintenance of the project ontology;
- identification of synonyms, ambiguities, and terminology gaps.

Corpus (D).

AI can support:

- automatic document classification;
- creation of semantic references linking document fragments to model elements;
- control of documentation completeness;
- extraction of structured data from unstructured text.

Population of the M, S, and D layers creates the information foundation on which the analytical Q layer operates.

16.3 Direction 3: Analytics and Forecasting — Layer Q

Layer Q forms the analytical part of the model, where information from M, S, and D is transformed into analytical results and forecasts.

This layer contains significant potential for AI-supported management and engineering decisions.

Applications include:

- forecasting schedule and cost based on graph structure, current readiness indicators, and historical data;
- risk assessment and management, including identification of critical paths, bottlenecks, cascading dependencies, and probabilistic scenario modeling;
- optimization of work sequences and commissioning-complex formation under resource constraints;
- anomaly and inconsistency detection, including deviations in data, inconsistencies between layers, and discrepancies with applicable requirements;
- integrated assessment of project status by combining indicators from different layers;
- semantic search across the document corpus using model context;
- generation of recommendations based on aggregated indicators, scenario analysis, and optimization.

16.4 Direction 4: Operational Support — Layer U

When the model is operating in support of an actual project, the meta-level coordinates all components.

Its functions include:

- **Real-time coordination.**

When new information becomes available, such as schedule updates, test results, or new documentation, the meta-level can recalculate Q metrics, verify consistency of M and S, and update semantic relationships.

- **Adaptive learning cycle.**

As data accumulate, the meta-level forms training datasets, retrain predictive models, evaluates their performance, and selects the most effective algorithms.

The cycle of data, training, evaluation, and improvement operates iteratively.

- **Generation and validation of new elements.**

When the model is expanded, every new element is checked for structural correctness, semantic consistency, and compliance with model invariants.

- **Model-quality monitoring.**

The system can monitor documentation completeness, classification consistency, forecast accuracy, and semantic-link coverage.

Degradation of any metric can indicate the need for corrective action.

- **Scenario modeling.**

Alternative model configurations can be evaluated through “what-if” scenarios, such as changing the sequence of activities, replacing equipment, or revising commissioning complexes, with consequences calculated across the model layers.

- **Algorithm replacement.**

A new algorithm can be connected through a standard interface, validated against historical data, and then introduced into the analytical workflow.

The four directions form a closed improvement cycle.

AI supports model development and implementation in Direction 1, model population in Direction 2, analytics in Direction 3, while the meta-level in Direction 4 coordinates the operation of all components.

The results of model use refine its structure, data, and algorithms, closing the continuous-improvement cycle.

17 Conclusion

The model is a digital twin of the NPP life cycle that integrates project management methods, ontological modeling, and modern artificial intelligence approaches into a unified and internally consistent framework.

Its principal distinction is that the model simultaneously:

- represents life-cycle processes without restrictions on their type or complexity;

- is structured for use by AI and machine learning algorithms;
- remains understandable to engineers, project managers, and domain experts.

The modeled object is not an individual physical asset but the life cycle as a whole, including its phases, dependencies, data, and decision-making processes.

Given the current development of artificial intelligence technologies, including graph neural networks and large language models, the life-cycle digital twin has significant potential as a foundation for intelligent management systems supporting nuclear power plant projects.

The architecture is designed to evolve together with advances in AI technologies.

It can scale in depth through increasingly detailed representation of the life cycle and in breadth through expansion of algorithms and functional capabilities.

At the same time, it can evolve together with the actual project and with the technologies used to analyze and manage it.

The model is a research development that can be adapted, when required, to specific tasks and projects.

Original publication

Yuriy Dmitrishin, *Formalization of a Nuclear Power Plant Life-Cycle Model for AI and Machine Learning Applications*, 2025.

DOI:

<https://doi.org/10.5281/zenodo.18527396>

GitHub repository:

<https://github.com/dmitrishin/npp-lifecycle-ai-ml-model>

License

Creative Commons Attribution 4.0 International (CC BY 4.0)

This summary is made available under the Creative Commons Attribution 4.0 International License. You are free to share and adapt the material, including for commercial purposes, provided that appropriate attribution is given to the author.

<https://creativecommons.org/licenses/by/4.0/>

Copyright © Yuriy Dmitrishin